

Pivot Tables

Python Data Science Handbook — Chapter 3, Section 3.9

A pivot table is a multidimensional groupby: aggregate one number across the grid formed by two categorical keys.

1 From one-dimensional grouping to a two-dimensional grid

Background & Context

In the previous section we met `groupby`: the *split-apply-combine* pattern. You pick a categorical column, split the rows into one bucket per category, apply a reducer (mean, sum, count, ...) to each bucket, and combine the per-bucket answers back into a single result indexed by the categories.

A `groupby` on *one* key gives you a one-dimensional summary: one row per category. But real questions are usually phrased along *two* axes at once. “What was the average sale *by region and by quarter*?” is not a list — it is a *table*: regions down the side, quarters across the top, an average in every cell. That two-dimensional cross-tabulation is exactly what a **pivot table** produces.

Intuition

A pivot table is a `groupby` on *two* keys at once, where one key is laid out along the rows and the other along the columns. Formally, it aggregates a chosen quantity over the *Cartesian product* of the two key sets: for every (row-category, column-category) pair, it collects the matching rows, runs a reducer on them, and drops the scalar result into the corresponding cell of a grid.

If you have ever built a pivot table in Excel or Google Sheets — dragging one field onto “Rows,” another onto “Columns,” and a numeric field onto “Values” with an aggregation like *Average* — you already know the mental model. Pandas’s `pivot_table` is the same idea expressed in code, with the full power of Python reducers behind it.

1.1 Why not just use `groupby` twice?

You can, in fact, reproduce a pivot table with the tools from §3.8. Group by *both* keys, aggregate, and then *unstack* the inner key from the row index up into the columns:

```
# long-form df with columns: region, quarter, sales
grouped = df.groupby(['region', 'quarter'])['sales'].mean()
table = grouped.unstack()    # move 'quarter' from rows to columns
```

This works, but it is clumsy: you spell out the grouping keys twice over, you have to remember which level `unstack` pulls up, and adding grand totals or a second reducer turns into a small project. `pivot_table` packages this whole pipeline behind one call with readable keyword arguments.

Key Idea

`df.pivot_table(values, index, columns, aggfunc)` is `df.groupby([index, columns])[values].aggfunc().unstack(columns)` — same computation, friendlier interface. Whenever a pivot table puzzles you, mentally rewrite it as the equivalent `groupby` and the result becomes obvious.

2 The shape of the operation

The diagram below shows the move from *long form* (one fact per row, the “tidy” layout) to the pivoted *wide grid*. Each long-form row carries a row-key (its region), a column-key (its quarter), and a value; the pivot routes that value into a cell, and where several rows land in the same cell the reducer collapses them to one number.

Long form (tidy: one fact per row)

region	quarter	sales = 12
region	quarter	sales = 18
region	quarter	sales = 9
region	quarter	sales = 15
region	quarter	sales = 14

Wide grid (region × quarter)

	Q1	Q2
North	13.0	18.0
South	9.0	15.0

two North/Q1 rows (12, 14) → mean 13.0

Intuition

Read the grid as a function of two arguments. The **row index** (blue) is the first key, the **columns** (green) are the second key, and each interior cell is the reducer applied to “all original rows whose first key matches this row label *and* whose second key matches this column label.” Empty combinations — pairs that never occur in the data — become `NaN` unless you tell Pandas otherwise.

3 A worked example, end to end

Let us build a small original dataset: a guest list for a workshop. Each attendee has a *track* they registered for, a *seniority* level, and we record their post-session *rating* of the workshop (0–10). The long-form table:

```
import numpy as np
import pandas as pd

guests = pd.DataFrame({
    'track':      ['Data', 'Data', 'Data', 'Systems', 'Systems',
                  'Systems', 'Data', 'Systems', 'Data', 'Systems'],
    'seniority':  ['Junior', 'Senior', 'Junior', 'Senior', 'Junior',
                  'Senior', 'Senior', 'Junior', 'Junior', 'Senior'],
    'rating':     [8, 9, 7, 6, 5, 8, 10, 4, 9, 7],
    'hours':      [3.0, 4.5, 2.0, 5.0, 1.5, 6.0, 4.0, 2.5, 3.5, 5.5],
})
```

guests

	track	seniority	rating	hours
0	Data	Junior	8	3.0
1	Data	Senior	9	4.5
2	Data	Junior	7	2.0
3	Systems	Senior	6	5.0
4	Systems	Junior	5	1.5
5	Systems	Senior	8	6.0
6	Data	Senior	10	4.0
7	Systems	Junior	4	2.5
8	Data	Junior	9	3.5
9	Systems	Senior	7	5.5

The natural two-dimensional question is: *what is the average rating for each (track, seniority) combination?* That is one call:

```
guests.pivot_table(values='rating', index='track', columns='seniority')
# aggfunc='mean' is the default, so it is implied here
```

seniority	Junior	Senior
track		
Data	8.0	9.5
Systems	4.5	7.0

Four numbers, each the mean of the matching original rows. The Data/Junior cell averages ratings 8, 7, 9 → 8.0; the Systems/Senior cell averages 6, 8, 7 → 7.0. The same answer, with no `unstack` to remember, comes from the explicit pivot syntax:

```
guests.pivot_table('rating', index='track', columns='seniority',
                    aggfunc='mean')
```

Key Idea

The default `aggfunc` is `'mean'`. If you forget to think about it, every cell is an average — which is the right default surprisingly often, but is also a classic source of confusion when you actually wanted a *sum* or a *count*.

3.1 Choosing the reducer, and using several at once

`aggfunc` accepts any reducer `groupby` accepts: a string name (`'sum'`, `'count'`, `'median'`, `'max'`), a NumPy function (`np.std`), or a list/dict for several at once. A *dict* lets you apply a different reducer to a different value column:

```
guests.pivot_table(index='track', columns='seniority',
```

```
aggfunc={'rating': 'mean', 'hours': 'sum'})
```

	hours		rating	
seniority	Junior	Senior	Junior	Senior
track				
Data	8.5	8.5	8.0	9.5
Systems	4.0	16.5	4.5	7.0

When `aggfunc` is a dict, you omit values — the dict keys already name the columns to aggregate. Notice the result now has a *hierarchical* column index: the outer level names the value column, the inner level the seniority.

3.2 Grand totals with margins

A spreadsheet pivot usually carries an “All” row and column holding the per-row, per-column, and overall aggregate. Set `margins=True`:

```
guests.pivot_table('rating', index='track', columns='seniority',
                    margins=True, margins_name='All')
```

seniority	Junior	Senior	All
track			
Data	8.00	9.5	8.60
Systems	4.50	7.0	6.00
All	6.25	8.2	7.30

Common Pitfall

The margin is the reducer applied to *all the underlying rows*, **not** the mean of the cells in that row or column. The grand-total All/All cell here is 7.30, the mean of all ten ratings. Averaging the four interior means instead would give $(8.0 + 9.5 + 4.5 + 7.0)/4 = 7.25$ — close, but wrong, because the four groups have different sizes. “Average of averages” is not the average; `margins` correctly recomputes from the raw data.

3.3 Filling and dropping empty cells

If a (row, column) pair never occurs, its cell is NaN. Two knobs control this:

- `fill_value=v` replaces missing cells with *v after* aggregation — handy for turning absent counts into 0.
- `dropna=True` (the default) drops any column whose every entry is NaN; set `dropna=False` to keep fully empty columns.

```
guests.pivot_table('rating', index='track', columns='seniority',
                    aggfunc='count', fill_value=0)
```

seniority	Junior	Senior
track		
Data	3	2
Systems	2	3

4 Multi-level pivot tables

The row and column arguments each accept a *list* of keys, producing a hierarchical (`MultiIndex`) grid. And — the trick that makes pivots powerful on messy data — a key need not be a pre-existing categorical column. You can bin a *continuous* variable on the fly and pivot on the bins.

4.1 Binning a continuous variable with `pd.cut` and `pd.qcut`

Our `hours` column is continuous, so it cannot index a grid directly — every value is its own “category.” We bucket it first:

- `pd.cut(x, bins)` splits by *value* into intervals you specify (equal-width or explicit edges). Buckets may hold very different counts.
- `pd.qcut(x, q)` splits by *quantile* into *q* groups of (roughly) equal *count*. Buckets have similar size but uneven width.

```
effort = pd.cut(guests['hours'], bins=[0, 3, 6], labels=['Light', 'Heavy'])
guests.pivot_table('rating', index=['track', effort],
                    columns='seniority', observed=False)
```

		seniority	Junior	Senior
track	hours			
Data	Light		7.5	NaN
	Heavy		9.0	9.5
Systems	Light		4.5	NaN
	Heavy		NaN	7.0

The row index is now hierarchical: `track` on the outer level, the `hours` bucket on the inner. NaNs appear where a combination is empty (no light-effort Senior in either track). Pass `observed=False` so that empty category combinations are still laid out — and remember `fill_value` if you would rather see a placeholder than NaN.

Python Refresher

`pd.cut` returns a *Categorical* object, the same kind of labeled-bucket array you get from `astype('category')`. Because it is categorical, Pandas knows the full set of buckets even when some are unused — which is why the empty “Light/Senior” combinations still get a row or column rather than vanishing. The `observed` flag chooses whether unused combinations are shown.

You can stack keys on the columns side too. Both axes can be lists, giving a fully four-way cross-tabulation:

```
guests.pivot_table('rating', index=['track', effort],
                    columns='seniority', aggfunc=['mean', 'count'],
                    observed=False, fill_value=0)
```

This nests mean and count as the outermost column level, each split again by seniority — a compact dashboard of both the average rating and how many people it was averaged over.

5 pivot_table VS groupby VS crosstab

These three overlap; choosing well is mostly about matching the tool to the question.

Tool	Best when you want...	Output shape
groupby	a 1-D summary, or a custom apply step	long (index = keys)
pivot_table	a 2-D summary of <i>one numeric column</i>	wide grid
crosstab	a 2-D <i>frequency count</i> of two categories	wide grid of counts

`pd.crosstab` is really `pivot_table` specialized to counting. These two calls are equivalent:

```
pd.crosstab(guests['track'], guests['seniority'])
guests.pivot_table(index='track', columns='seniority',
                    aggfunc='size', fill_value=0)
```

seniority	Junior	Senior
track		
Data	3	2
Systems	2	3

Key Idea

Reach for `crosstab` when the cells are just *how many* (a contingency table); reach for `pivot_table` when the cells are an aggregate of some *measured quantity*; and drop back to plain `groupby` when you need one dimension only, or a bespoke `.apply` that no built-in reducer expresses. They are three doors into the same split-apply-combine engine.

Section recap

- A pivot table is a *two-key groupby*: it aggregates one value over the Cartesian product of a row key and a column key, producing a grid.
- `pivot_table(values, index, columns, aggfunc)` equals `groupby([index, columns])[values].aggfunc().unstack()` — the same computation with a cleaner interface; the Excel “Rows / Columns / Values” model is the right intuition.
- Default `aggfunc` is `'mean'`; pass a string, function, list, or dict (dict applies a different reducer per value column).
- `margins=True` adds correct grand totals recomputed from raw data — never an average of the

cell averages.

- `fill_value` replaces empty cells; `index/columns` accept lists for hierarchical grids; `pd.cut/pd.qcut` turn a continuous column into a usable key.
- Use `crosstab` for frequency counts, `pivot_table` for aggregated measurements, and plain `groupby` for 1-D or custom summaries.